* Unit 2 *
Graphics Primitives

Graphics primitives are the basic geometric elements used to construct graphical objects & images.
→ Big objects are combination of shapes & elements. (Complex)

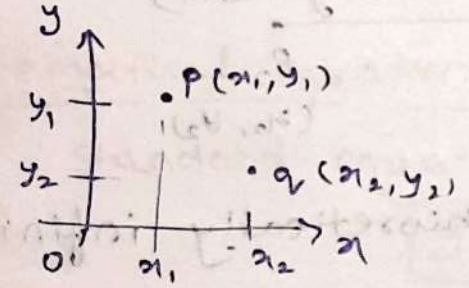
Common graphic primitives

→ Point
→ lines
→ Circle
→ Ellipse
→ Polygon
→ character/Text.

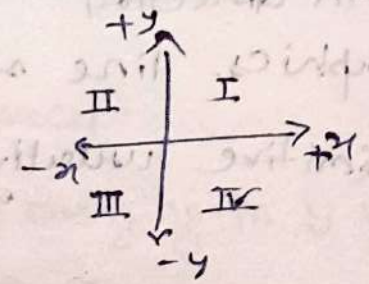
* Point :-

A point is the simple graphic primitive & represents a single position in a coordinate system. Represented using coordinates $p(x, y)$.

Ex



x → horizontal position
y → vertical position.
0 ⇒ origin $(0, 0)$



$(x, y) \neq (y, x)$

~~Line~~

Raster display → The system ultimately needs to determine which pixel position should be activated/displayed.

→ points are premivitive because points connects to make lines.

point in 2D

$P = (x, y)$

x → horizontal

y → vertical

Points in 3-D

$P(x, y, z)$

x → horizontal

y → vertical

z → depth

Important Algorithm

↳ DDA Algo.

↳ Bresenham Algo.

* Line

A line is a straight geomatric primitive that connects two points in a coordinate system.

$P_1(x_1, y_1)$

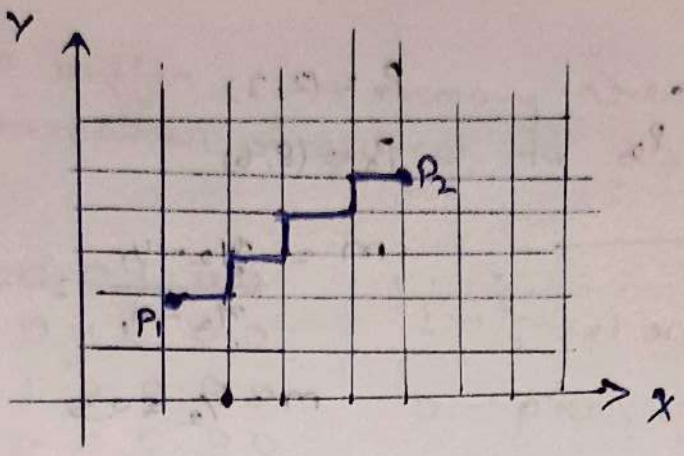
$P_2(x_2, y_2)$



→ In mathematics, line theoretically infinite in both direction

but in graphics, line segement have two endpoints

4 Hae primitive usually represents a line segement



+ Computer decides which pixel is to be connected to make line segement between two endpoints.

Note Computer graphics have grids (pixels), so we can not draw direct continuous line.
Line drawing Algorithms are used to select suitable pixel to represent a line.

- Scan conversion of a line

Scan conversion is the process of converting a geometric object such as line into a set of pixels that can be displayed on a raster screen.

Mathematical equation of line

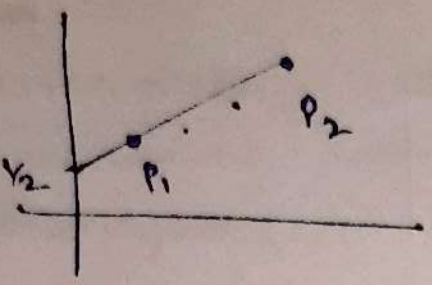
Standard equation of a straight line is

$$y = mx + c$$

m = Slope
c = y-Intercept

Slope => rate of changing in y with respect to changing in x.

$$m = \frac{\Delta y}{\Delta x} = \frac{y_2 - y_1}{x_2 - x_1}$$



$P_1 = (2, 3)$

$P_2 = (8, 6)$

$$m = \frac{y_2 - y_1}{x_2 - x_1} = \frac{6-3}{8-2} = \frac{3}{6}$$

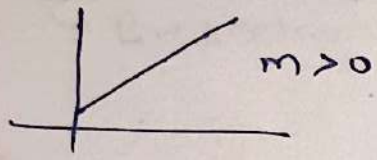
$$m = 1/2 = 0.5$$

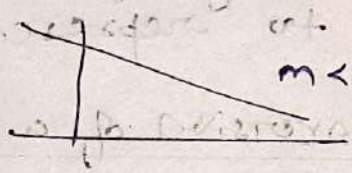
$$y = \frac{1}{2}x + c$$

$$\boxed{2y - x = c_1} \qquad c = 1$$

$$\boxed{2y - x = 1}$$

Positive & Negative slope

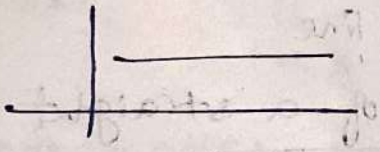




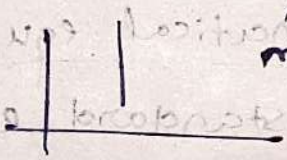
line rises from left to right

line falls from left to right.

If $m = 0$ then $\Delta y = 0$



zero slope



undefine slope

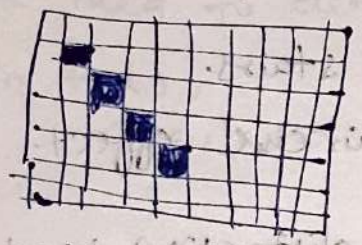


→ Frame buffer is a memory area that stores the information required to display an image on screen.

```
0 0 0 1 0 0
0 0 1 0 0 0
0 1 0 0 0 0
1 0 0 0 0 0
0 1 0 0 0 0
0 0 1 0 0 0
0 0 0 1 0 0
```

1 → pixel on

0 → pixel off

→ 

Each pixel is represented as a cell.

It shows staircase line

But Algorithm decide which pixels should be switched ON so that the selected pixels look like a line?

→ We need intermediate points to generate a line.

→ Line generation is the process of determining the pixels that should be illuminated to represent a line between two given endpoints on a raster display.

Note we cannot directly use standard line equation, because it may give points values like

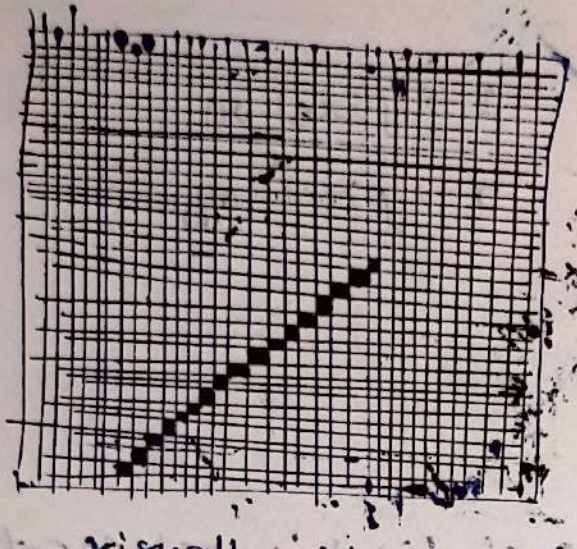
(3,3) (3, 3·5) (4,4), (5, 4·5) ....

but pixel are discrete (Integer)

So it required Algorithm

* Staircase Effect

When a diagonal line is represented using discrete pixels, it doesn't appear prefectly smooth.



visually it resembles stairs.

This is known as staircase effect.

→ Staircase effect is the distortion produced when a smooth mathematical line is approximated using discrete pixels on a raster display, causing the line to appear like a staircase.

DDA Algorithm of line

(Digital Differential Algorithm)

DDA is a line generation algorithm that generates a line by calculating intermediate points between two given endpoints & selecting the pixels closest to those calculated positions.

Let suppose we have two end points

(x_0, y_0) & (x_1, y_1)

we need to determine intermediate points.

-> In this,

Calculate slope (deviation)

$$m = \frac{y_1 - y_0}{x_1 - x_0}$$

$$\boxed{m = \frac{dy}{dx}}$$

DDA has three main cases -

Case 1 $0 < m < 1$

$dy < dx$

x changes faster than y.

So increment x by 1

$$\boxed{x_{i+1} = x_i + 1}$$

then $y_{i+1} = y_i + m$

(use rounding to select y)

Case 2 $m \geq 1$

$dx < dy$

y changes faster than x

So increment y by 1

$$\boxed{y_{i+1} = y_i + 1}$$

then $\boxed{x_{i+1} = x_i + 1/m}$

(here take round(x))

Case III

If slope $m = 1$

$dx = dy$

rate of x & y is equal.

~~$x_{i+1} = x_i$~~

So increment x & y by 1.

$$x_{i+1} = x_i + 1$$
$$y_{i+1} = y_i + 1$$

$(\theta = 45°)$

Algorithm start consider start & ending point.

```
dx = x1 - x0
dy = y1 - y0          (abs => absolute)

steps = abs(dx) > abs(dy) ? abs(dx) : abs(dy);

xi = dx/(float)steps;
yi = dy/(float)steps;

x = x0;
y = y0;

for(int i=0; i<=steps; i++)
{
    putpixel(round(x), (round(y), WHITE);
    x += xi;
    y += yi;
}
```

* Algorithm *

```
x = x0
y = y0
m = (y1 - y0) / (x1 - x0)
if ( m > 1 )
    {
    m1 = 1/m
    while y <= y1 ;
        putpixel (round(x), y)
        y = y + 1 ;
        x = x + m1
    }
else if (.0 < m < 1)
    {
    while x <= x1 ;
        putpixel ( x, round(y) )
        x = x + 1 ;
        y = y + m;
    }
```

Q ⇒ $P_0 = (2, 4)$ $dx = 9 - 2 = 7$

$P_1 = (9, 9)$ $dy = 9 - 4 = 5$ $m = 5/7$

$m < 1$

Case 1 Applied

$x_{i+1} = x_i + 1$ ⇒

$y_{i+1} = y_i + m$

So first point,

$(x_0, y_0) = (2, 4)$

$x_1 = 2 + 1 = 3$

$y_1 = 4 + 5/7 \approx 5$

$(x_1, y_1) = (3, 5)$

Now Next

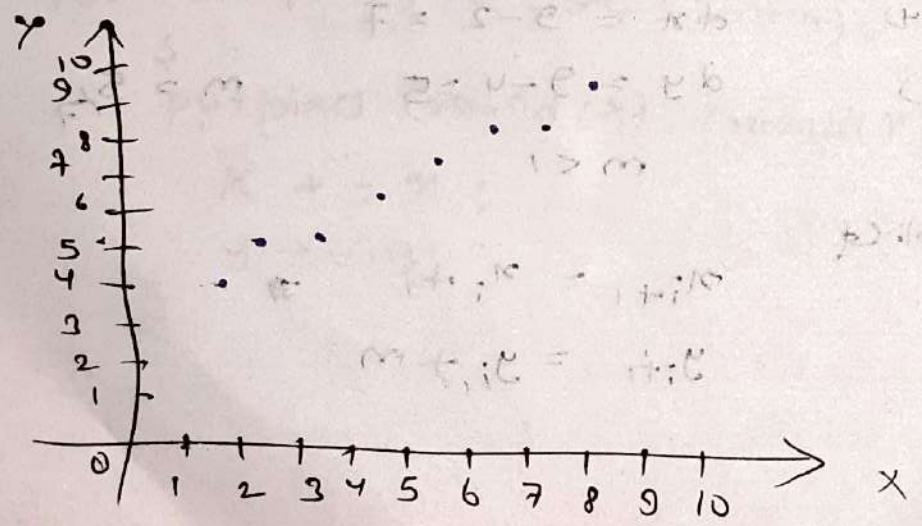
$x_2 = 3 + 1 = 4$

$y_2 = 5 + 5/7 = 5$

$(x_2, y_2) = (4, 5)$

Similarly

Step	X	Y
0	2	4
1	3	5
2	4	5
3	5	6
4	6	7
5	7	8
6	8	8
7	9	9



Advantage

1. Simple
2. Easy incremental calculation
3. Works for different slopes.

Disadvantage

1. Floating point arithmetic involved
2. Rounding required.
   round().
3. Slower then Bresenham.

Q Draw a line segement from point (2,4) to (9,9) using DDA Algorithm

Sol Given points P_1 (2,4)

P_2 (9,9)

$\Delta P = P_2 - P_1$

$\Delta x = 9-2 = 7$

$\Delta y = 9-4 = 5$

Here $\Delta x > \Delta y$ then $x_{i+1} = x_i + 1$

$0 < m < 1$

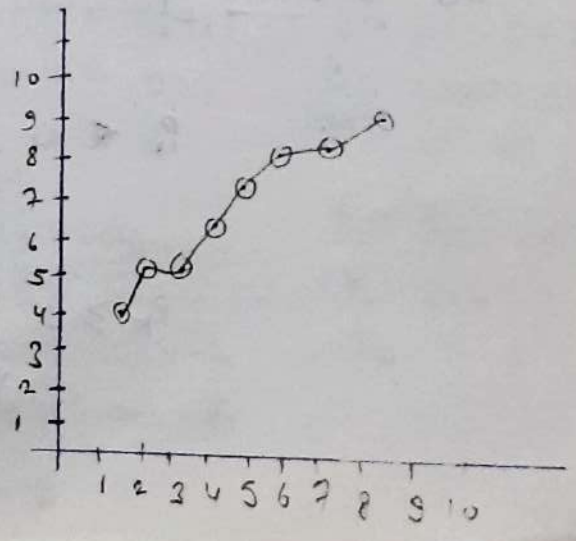
$y_{i+1} = y_i + m$

$m = \frac{\Delta y}{\Delta x} = 5/7 = 0.71$

$y = mx + c$

$y = 5/7 x + 18/7$

x_i	y_i	x_{i+1}	y_{i+1}	Points
2	4	3	4.71	(3,5)
3	4.71	4	5.42	(4,5)
4	5.42	5	6.13	(5,6)
5	6.13	6	6.84	(6,7)
6	6.84	7	7.55	(7,8)
7	7.55	8	8.26	(8,8)
8	8.26	9	8.98	(9,9)



* Bresenham's Line Algorithm :-

Bresenham's line algorithm is an incremental line drawing algorithm used for scan conversion of a line. It uses adition, substraction & multiplication operations to determine the appropriate pixels for drawing a line.

Note
= It determine next pixel using a decision parameter & mainly uses integer arithmetic (P_k).

Case 1
=
$0 < m < 1$

$\Delta x > \Delta y$

then

$$\left\{ \begin{array}{l} x_{k+1} = x_k + 1 \\ y_k = y_k \text{ or } y_{k+1} \end{array} \right\}$$

y_k → east, y_{k+1} → North-east

decision Parameter

$$\boxed{P_0 = 2\Delta y - \Delta x}$$

Case 1

P_k decides what y_k will be.

if

$P_k < 0$

$(x_k + 1, y_k)$

and

$$\boxed{P_{k+1} = P_k + 2\Delta y}$$

Case 2
=
$P_k \geq 0$

$(x_k + 1, y_k + 1)$

$$\boxed{P_{k+1} = P_k + 2\Delta y - 2\Delta x}$$

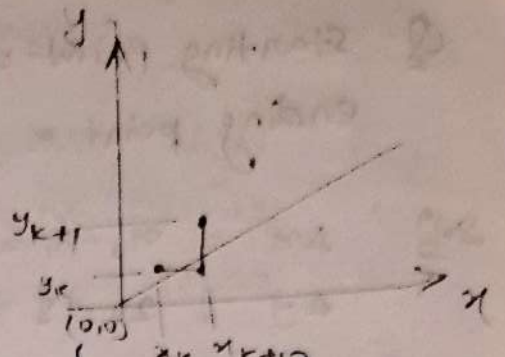
So Summary

$$P_k < 0 \left\{ \begin{array}{l} x_{k+1} = x_k + 1 \\ y_{k+1} = y_k \\ P_{k+1} = P_k + 2\Delta y \end{array} \right.$$

$$P_k \geq 0 \left\{ \begin{array}{l} x_{k+1} = x_k + 1 \\ y_{k+1} = y_k + 1 \\ P_{k+1} = P_k + 2\Delta y - 2\Delta x \end{array} \right.$$

Algorithm

1. Start
2. Starting point (x_1, y_1)
   ending point (x_2, y_2)
3. Calculate Δx, Δy, m
   $\Delta x = x_2 - x_1$
   $\Delta y = y_2 - y_1$
   $m = \frac{\Delta y}{\Delta x}$



4. ($P_0 = 2\Delta y - \Delta x$)
5. Check decision parameter P_k.

   if $P_k < 0$

   (x_{k+1}, y_k)

   $P_{k+1} = P_k + 2\Delta y$

   else $(P_k \geq 0)$

   (x_{k+1}, y_{k+1})

   $P_{k+1} = P_k + 2\Delta y - 2\Delta x$
6. Repeat until ending point.

   ($\Delta x + 1$ total iteration)

Ex: we have starting point = (9, 18)
ending point = (14, 22)

$\Delta x = 14 - 9 = 5$
$\Delta y = 22 - 18 = 4$
$m = 4/5 < 1$ ✓

$P_0 = 2\Delta y - \Delta x$
$P_0 = 2 \times 4 - 5$
$P_0 = 3$

x_k	y_k	P_k	x_{k+1}	y_{k+1}	P_{k+1}
9	18	3	10	19	$3 + 2\times4 = 2\times5 = 1$
10	19	1	11	20	$1 + 2\times4 - 2\times5 = -1$
11		-1	12		$-1 + 2\times4 - 2\times5 = -3$
12		-3	13		$-3 + 2\times4 - 2\times5 = -5$
13		-5	14		-7

Q Starting point = (9,18) ending point = (14,22) used Bresenham Algorithm.

Sol $\Delta x = x_2 - x_1 = 14 - 9 = 5$

$\Delta y = 22 - 18 = 4$

$m = 4/5 = 0.8$ ($m < 1$ ✓)

$P_0 = 2\Delta y - \Delta x = 8 - 5$

$P_0 = 3$

$P_0 \geq 0$ Case 1

$\Delta x - 1 = 5 - 1$

$\Rightarrow 4$ iterations

① $P_0 \geq 0$

$x_1 = x_0 + 1 = 9 + 1 = 10$

$y_1 = y_0 + 1 = 18 + 1 = 19$

$P_1 = P_0 + 2\Delta y - 2\Delta x$

$= 3 + 2 \times 4 - 2 \times 5 = 1$

$P_1 = 1$ & point (10,19)

② $P_1 = 1 \geq 0$

$x_2 = x_1 + 1 = 10 + 1 = 11$

$y_2 = y_1 + 1 = 19 + 1 = 20$

$P_2 = P_1 + 2\Delta y - 2\Delta x$

$= 1 + 2 \times 4 - 2 \times 5 = -1$

$P_2 = -1$ & point (11,20)

③ $P_2 = -1 < 0$

$x_3 = x_2 + 1 = 11 + 1 = 12$

$y_3 = y_2 = 20$

$P_3 = P_2 + 2\Delta y$

$P_3 = -1 + 8$

$P_3 = 7$ & point (12,20)

④ $P_3 = 7 \geq 0$

$x_4 = x_3 + 1 = 12 + 1 = 13$

$y_4 = y_3 + 1 = 20 + 1 = 21$

$P_4 = P_3 + 2\Delta y - 2\Delta x$

$P_4 = 7 + 8 - 10$

$P_4 = 5$ & (13,21)

⑤ Now $P_4 = 5 \geq 0$

$x_5 = x_4 + 1 = 13 + 1 = 14$

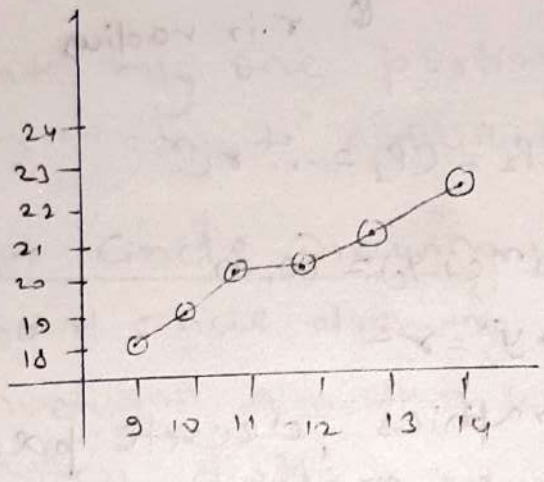
$y_5 = y_4 + 1 = 21 + 1 = 22$

(14,22) last point

NE = North East
E = East

So Table is

Step	Current	P_k	Decision	Next	P_{k+1}
0	(9,18)	3	$P \geq 0$ NE	(10,19)	1
1	(10,19)	1	$P \geq 0$ NE	(11,20)	-1
2	(11,20)	-1	$P < 1$ E	(12,20)	7
3	(12,20)	7	$P \geq 0$ NE	(13,21)	5
4	(13,21)	5	$P \geq 0$ NE	(14,22)	3rd



Note If $m > 1$

$P_0 = 2\Delta x - \Delta y$

$P_k < 0$

$x_{k+1} = x_k$

$y_{k+1} = y_k + 1$

$P_{k+1} = P_k + 2\Delta x$

$P_k \geq 0$

$x_{k+1} = x_k + 1$

$y_{k+1} = y_k + 1$

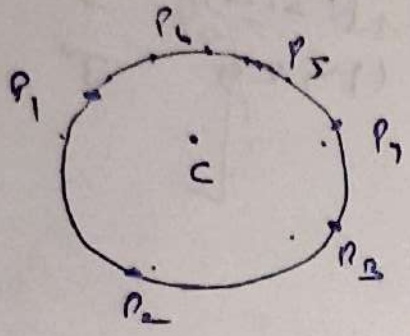
$P_{k+1} = P_k + 2\Delta x - 2\Delta y$

* Circle

A circle is a set of all points in a plane that are at a fixed distance from a fixed point.

fixed point (centre)

$$(x - x_c)^2 + (y - y_c)^2 = r^2$$



here

C (x_c, y_c) is center

& r is radius

$$CP_1 = CP_2 = CP_3 = \dots r$$

with origin $(x_c, y_c) = (0, 0)$

So $x^2 + y^2 = r^2$

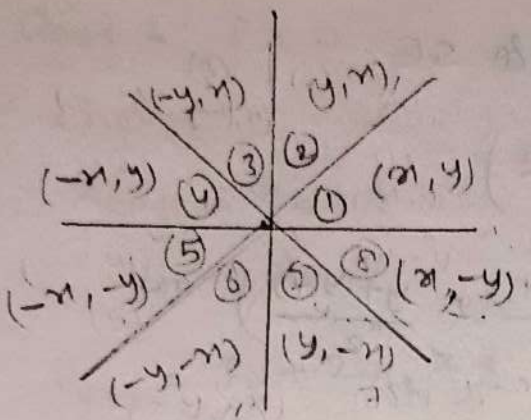
In Computer graphics, decrete pixels. Means Approximate circle.

Circle Scan Conversion

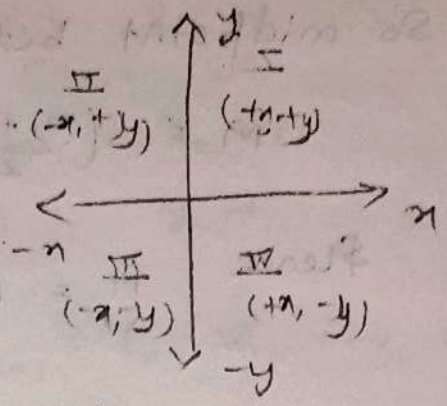
It is a process of converting a mathematical circle into set of points (pixels) that can be displayed on a raster screen.

Algorithm used -

* MidPoint Circle Algorithm
* Bresenham circle Algorithm.



(8-way Symmetry)



(4-way Symmetry)

Calculate only one portion of the circle & use symmetry to obtain the remaining portions.

* Midpoint Circle Drawing Algorithm

Midpoint circle drawing Algorithm is an incremental scan-conversion Algorithm used to draw a circle on a raster display by selecting the pixel closest to the actual mathematical circle.

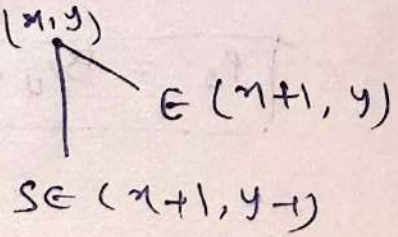
It uses
- decision Parameter
- Incremental calculation
- Circle symmetry
- Mainly integer arithmetic

decision Parameter => E (east)
or SE (South east)

P(x, y)

East = (x+1, y)

South East = (x+1, y-1)



So midpoint between E & SE

$$M = \left(\frac{x_1 + x_2}{2}, \frac{y_1 + y_2}{2}\right)$$

Hence $$M = \left(\frac{(x+1) + (x+1)}{2}, \frac{y + (y-1)}{2}\right)$$

$$M = (x+1, y - 1/2)$$

So circle function

$$f(M) = x^2 + y^2 - r^2$$

Hence

$$f(M) = (x+1)^2 + (y-1/2)^2 - r^2$$

Case 1	Case 2	Case 3
$f(M) < 0$	$f(m) = 0$	$f(m) > 0$
midpoint inside the circle	midpoint lies on circle	midpoint is outside of circle
Select SE for P	Select E for P	select E for P

* Initial Decision Parameter

starting Point $(x_0, y_0) = (0, r)$

$$\boxed{M = (1, r - 1/2)}$$

$$P_0 = f(1, r - 1/2)$$

$$\Rightarrow P_0 = 1^2 + (r - 1/2)^2 - r^2$$

$$\boxed{P_0 = 5/4 - r}$$

Case 1 $P < 0$

Choose E

$x_{k+1} = x_k + 1$

$y_{k+1} = y_k$

$P_{k+1} = P_k + 2x_k + 1$

Case 2 $P \geq 0$

Choose SE

$x_{k+1} = x_k + 1$

$y_{k+1} = y_k - 1$

$P_{k+1} = P_k + 2x_k - 2y_k + 1$

So Algorithm

1. Start
2. Enter center (x_c, y_c) & radius r
3. Initialize $x = 0$
   $y = r$
4. Initialize decision parameter
   $P_0 = 1 - r = 5/4 - r$
5. Plot the eight symmetric point of (x, y)
6. if
   $P < 0$
   $x = x + 1$
   $P = P + 2x + 1$

   $P \geq 0$ then
   $x = x + 1$
   $y = y - 1$
   $P = P + 2x - 2y + 1$
7. repeat step 5 - 7 until
   $x \geq y$
8. Stop.

Q Given a circle, radius is 10, determine the position along the circle of tense in first quadrant $x=0$ to $x=y$

Sol^n

$x_0 = 0$ to y

$y = r$ to 0

$P_0 = 1 - r = 1 - 10 = -9$

$\boxed{P_0 = -9}$

$P_k < 0$	$P_k \geq 0$
$x_{k+1} = x_k + 1$	$x_{k+1} = x_k + 1$
$y_{k+1} = y_k$	$y_{k+1} = y_k - 1$
$P_{k+1} = P_k + 2x_k + 1$	$P_{k+1} = P_k + 2x_k - 2y_k + 1$

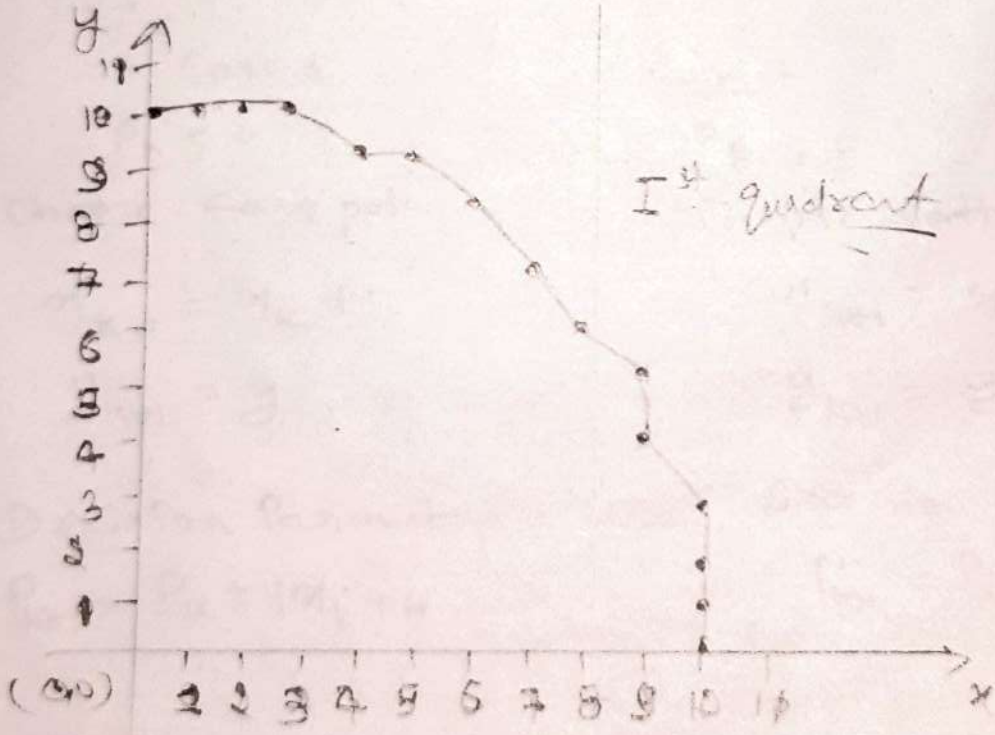
x_k	y_k	P_k	Point (x_k, y_k)
Initial 0	10	-9	(0, 10)
0+1=1	10	$-9+2\times1+1=-6$	(1, 10)
1+1=2	10	$-6+2\times2+1=-1$	(2, 10)
2+1=3	10	$-1+2\times3+1=6$	(3, 10)
3+1=4	10−1=9	$6+2\times4-2\times9+1=-3$	(4, 9)
4+1=5	9	$-3+2\times5+1=8$	(5, 9)
5+1=6	9−1=8	$8+2\times6-2\times8+1=5$	(6, 8)
6+1=7	8−1=7	$5+2\times7-2\times7+1=-6$	(7, 7)
			End

(x, y)

for first quadrant.

form $(x=0, y=0)$ $(x \to y$ & $y \to x)$

(0, 10) & (10, 0)
(1, 10) & (10, 1)
(2, 10) & (10, 2)
(3, 10) & (10, 3)
(4, 9) & (9, 4)
(5, 9) & (9, 5)
(6, 8) & (8, 6)
(7, 7) center point.



Bresenham Circle Algorithm:-

It is an incremental scan-conversion algorithm used to draw a circle on a raster display using integer arithmetic (a decision parameter.

-> It uses integer arithemetic
-> Avoids floating-point calculations.
-> No square-root calculation
-efficient for raster displays
-> Uses symmetry to reduce calculation.

Start point => $x = 0$
$y = r$ (r is radius).

then $(x, y) = (0, r)$

Decision parameter $P_0 = 3 - 2r$ (initial)

Cases

Case 1

$P_k < 0$

Choose East point

$x_{k+1} = x_k + 1$

$y_{k+1} = y$

Decision Parameter

$P_{k+1} = P_k + 4x_k + 6$

Case 2

$P_k \geq 0$

Choose North east point

$x_{k+1} = x_k + 1$

$y_{k+1} = y_k - 1$

Decision parameter.

$P_{k+1} = P_k + 4(x_k - y_k) + 10$

Algorithm

1. Start
2. Enter the center (x_c, y_c) and radius r.
3. Initialize $x = 0$
   $y = r$
4. Calculate initial decision parameter
   $P_0 = 3 - 2r$
5. Plot the eight symmetric point of (x, y)
6. If

$P < 0$	$P \geq 0$
$x \Rightarrow x+1$	$x \rightarrow x+1$
$P \rightarrow P + 4x + 6$	$y \rightarrow y-1$
	$P \rightarrow P + 4(x-y) + 10$

6. Plot the eight symmatic points of the new (x, y)
7. Repeat until $x \geq y$
8. Stop.

Q Given the center point coordinate $(0, 0)$
radius = 8

Soln Given $r = 8$

Initial decision Parameter

$$P_0 = 3 - 2r$$
$$= 3 - 16 = -13$$

$$\boxed{P_0 = -13}$$

$$\begin{pmatrix} x_0 = 0 \\ y_0 = 8 \end{pmatrix}$$

Previous (x, y)	P_k	Point (x, y)	Next P_{k+1}
$(0, 0)$	-13	$(0, 8)$	$P_1 = -13 + 4 \times 0 + 6 = -7$
$(0, 8)$	-7	$(0+1, 8) = (1, 8)$	$P_2 = -7 + 4 \times 1 + 6 = 3$
$(1, 8)$	3	$(1+1, 8), (2, 8)$	$P_3 = 3 + 4(2-8) + 10 = -11$
$(2, 8)$	-11	$(2+1, 8-1) = (3, 7)$	$P_4 = -11 + 4 * 3 + 6 = 7$
$(3, 7)$	7	$(3+1, 7-1) = (4, 7)$	$P_5 = 7 + 4(4-7) + 10 = 5$
$(4, 7)$	5	$(4+1, 7-1) = (5, 6)$	$P_6 =$ No Need $(x > y)$

So the Points $(0,8)$, $(1,8)$, $(2,8)$, $(3,7)$, $(4,7)$ & $(5,6)$

Complete 4 Quadrant Points

Quadrant I $(+x, +y)$

$(0,8)$, $(8,0)$
$(1,8)$, $(8,1)$
$(2,8)$, $(8,2)$
$(3,7)$, $(7,3)$
$(4,7)$ & $(7,4)$
$(5,6)$ & $(6,5)$

Quadrant II $(-x, +y)$

$(0,+8)$, $(-8,0)$
$(-1,8)$, $(-8,1)$
$(-2,8)$, $(-8,2)$
$(-3,7)$, $(-7,3)$
$(-4,7)$, $(-7,4)$
$(-5,6)$, $(-6,5)$

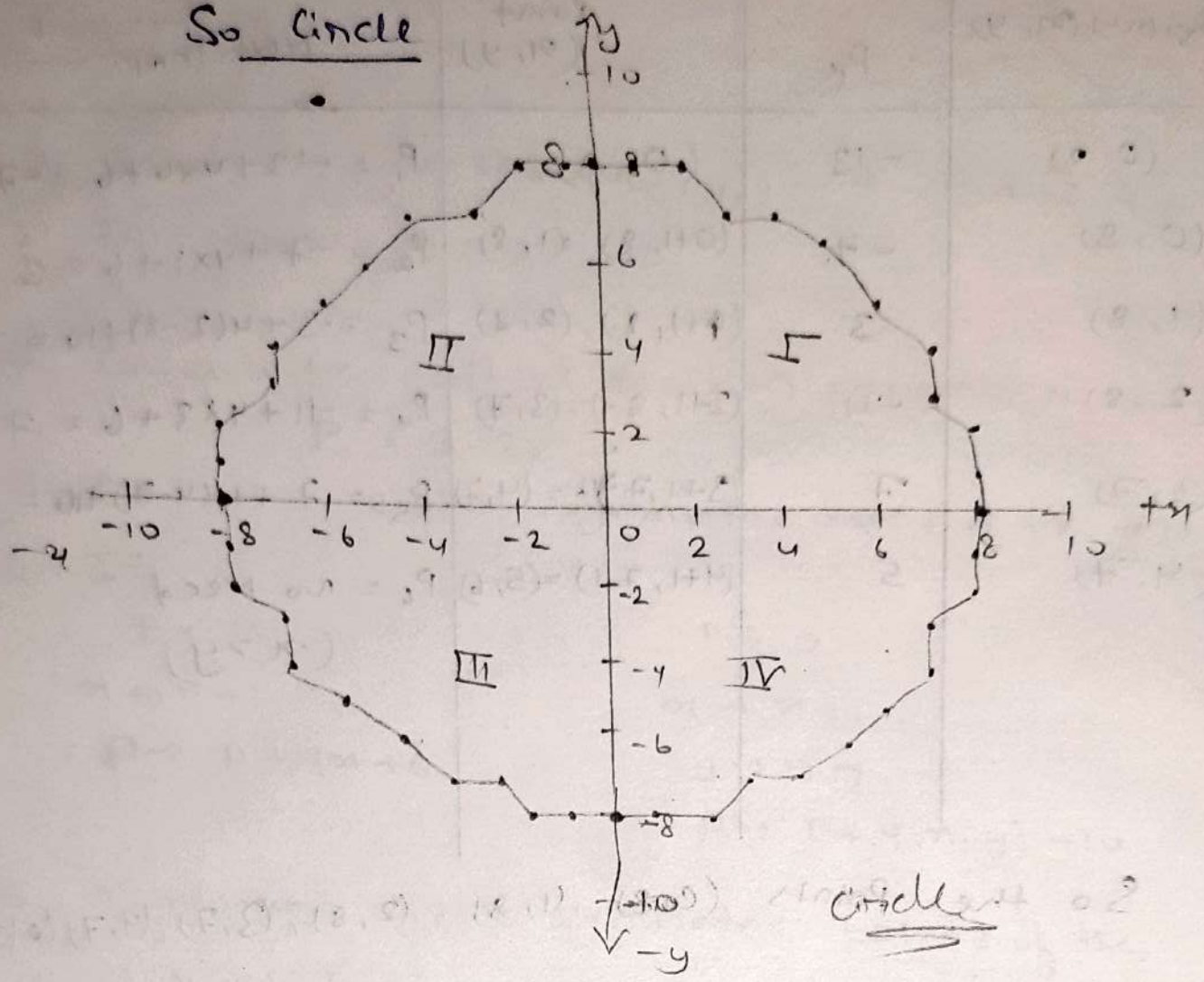
Quadrant III $(-x, -y)$

$(0,-8)$, $(-8,0)$
$(-1,-8)$, $(-8,-1)$
$(-2,-8)$, $(-8,-2)$
$(-3,-7)$, $(-7,-3)$
$(-4,-7)$, $(-7,-4)$
$(-5,-6)$, $(-6,-5)$

Quadrant 4 $(+x, -y)$

$(0,-8)$, $(+8,0)$
$(1,-8)$, $(+8,-1)$
$(2,-8)$, $(8,-2)$
$(3,-7)$, $(7,-3)$
& $(4,-7)$, $(7,-4)$
$(5,-6)$, $(6,-5)$

So Circle



Circle

* Ellipse

An ellipse is a set of all points in a plane such that the sum of their distance from two fixed points, called foci, is constant.

So $d_1 + d_2 =$ constant.

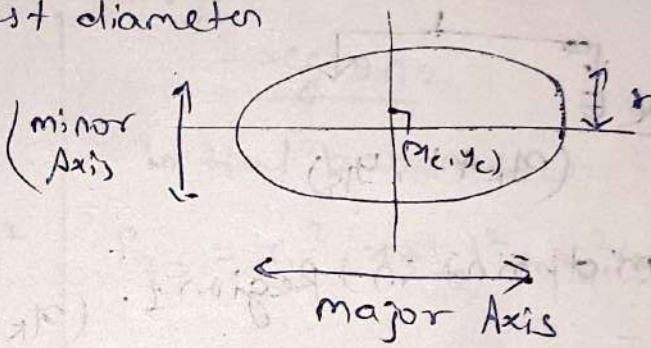
Let a point P on ellipse

foci = F_1 & F_2

the $PF_1 + PF_2 =$ Constant.

major Axis => longest diameter

minor axis => shortest diameter



equation

$$\frac{(x-x_c)^2}{r_x^2} + \frac{(y-y_c)^2}{r_y^2} = 1$$

r_x = semi major

r_y = semi minor

for center at origin

$$\frac{x^2}{r_x^2} + \frac{y^2}{r_y^2} = 1$$

Parametric equation $x = x_c + r_x \cos\theta$

$y = y_c + r_y \sin\theta$

It has 4 way Symmetry. only.

So (x, y), $(-x, y)$, $(-x, -y)$ & $(x, -y)$

Mid Point Algorithm for Ellipse

Incremental Algorithm used to draw an ellipse on a raster scan display by selecting the appropriate pixel using a decision parameter

Starting point $x_0 = 0$

$y_0 = r_y$

$(x_0, y_0) = (0, r_y)$

It has two region. (m = magnitude of slope)

$|m| < 1$ — x is incremented

$|m| \geq 1$ (more ~~se~~) — y is decremented.

Choose E

(x_{k+1}, y_k) (x_{k+1}) $(y_k - 1)$

midpoint in Region 1. $(x_k + 1, y_k)$ & $(x_k + 1, y_k - 1)$

midpoint $(x_k + 1, y_k - 1/2)$

ellipse function $f(x,y) = r_y^2 x^2 + r_x^2 y^2 - r_x^2 r_y^2$

if $f(x,y) < 0$ — point inside ellipse

$f(x,y) = 0$ — point on ellipse

$f(x,y) > 0$ — point outside of ellipse.

Decision Parameter

midPoint $= (x_k + 1, y_k - 1/2)$

$$P_{1k} = r_y^2 (x_k + 1)^2 + r_x^2 \left(y_k - \tfrac{1}{2}\right)^2 - r_x^2 r_y^2$$

if Case 1 | Case 2

$P_k < 0$ | $P_k \geq 0$

$x \to x+1$ | $x \to x+1$

$y \to y$ | $y \to y-1$

$P_{k+1} = P_k + 2r_y^2 x_{k+1} + r_y^2$ | $P_{k+1} = P_k + 2r_y^2 x_{k+1} + r_y^2 - 2r_x^2 y_{k+1}$

Initial decision parameter

$$P_{10} = r_y^2 - r_x^2 r_y + \frac{1}{4} r_x^2$$

when $2r_y^2 x \geq 2r_x^2 y$ then move to next region.

Start $(0, r_y)$

region 1

initial parameter

$P_{10} = r_y^2 - r_x^2 r_y + \frac{1}{4} r_x^2$

Condition

$2r_y^2 x \leq 2r_x^2 y$

if $P_1 < 0 \Rightarrow (x+1, y)$

$P_1 \geq 0 \Rightarrow (x+1, y-1)$

region 2

initial parameter

$P_{20} = r_y^2 (x+1/2)^2 + r_x^2 (y-1)^2 - r_x^2 r_y^2$

if $P_2 > 0 \Rightarrow (x, y-1)$

$P_2 \leq 0 \quad (x+1, y-1)$

Q Determine pixels position of an ellipse with radii $r_x = 6$ & $r_y = 10$ at centered (0,0). with midpoint algorithm

Soln

Constant $r_x^2 = 36$, $r_y^2 = 100$

$2r_x^2 = 72$ $\quad 2r_y^2 = 200$

Region 1 (Slope < 1)

Start at $(0, r_y)$ (0,10)

Initial Parameter

$P_{10} = r_y^2 - r_x^2 \cdot r_y + \frac{1}{4} r_x^2 = 100 - 360 + 9 \Rightarrow -251$

(Condition $2r_y^2 x \geq 2r_x^2 y$)

k	P_{1k}	Pixel (x, y)	$2r_y^2 x$	$2r_x^2 \cdot y$
0	-251	(0,10)	0	72×10 = 720
1	-251+100+100 = -49	(1,10)	200	720
2	-49+200+100 = 99	(2,9)	400	648
3	601	(3,9)	600	648

$2r_y^2 x = 800$ & $2r_x^2 y = 576$ so

region 2

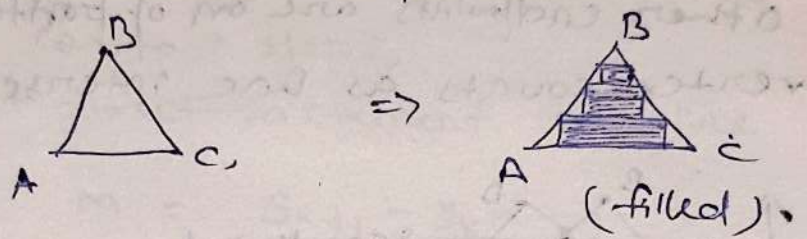
Start at 3+1 & 9-1 = (4,8)

$P_{20} = r_y^2 (x + \frac{1}{2})^2 + r_x^2 (y-1)^2 - r_x^2 r_y^2 = 189$

Iteration	P_{2k}	(x, y)
0		
1		

* Filled-Area Primitives :-

A filled area primitive is a closed graphical region whose interior pixels are filled with a specified color or pattern.



used in Buildings, Cars, Characters, Maps, 2D games, Computer-aided design etc

In Raster system,

Images are made of pixel in Raster display. So It is not mathematically filled. So identify interior pixels → set their Colors.

Approaches

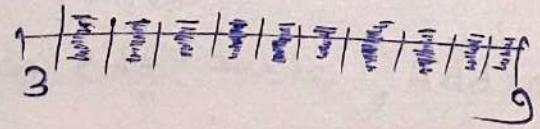
1) Scan-line filling -

polygon is intersect using different horizontal scan lines.

i) intersection with polygon edges
ii) Arrange left to right intersections point
iii) Fill pixels between intersection pair.

$x_1 = 3$

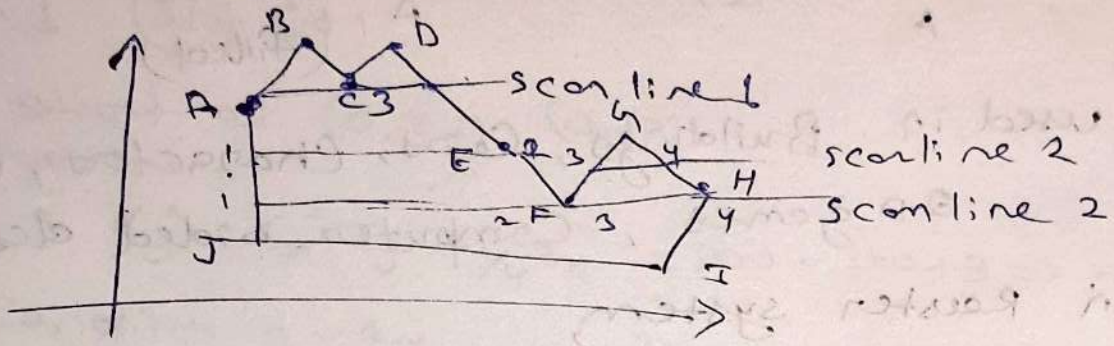
$x_2 = 9$



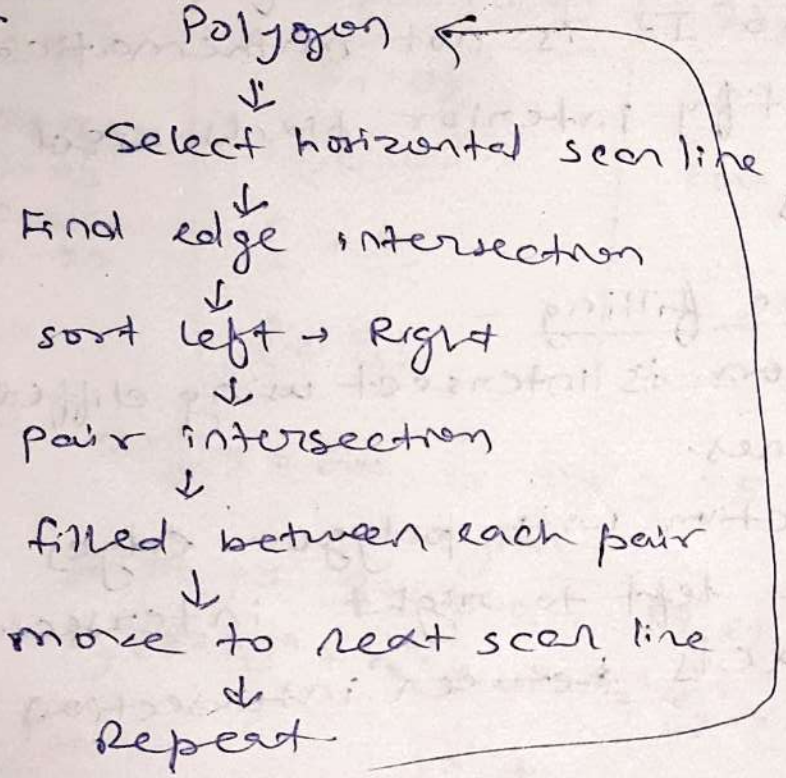
If scan line intersects at vertex point.

Case 1 other end point same side
vertex counts as Two intersections

Case 2 other endpoints are on opposite side
vertex counts as One intersection.



Overview



Coherence Method

use incremental calculation along a scan line or between successive scan lines to efficiently determine polygon-edge intersections due to constant slope,

just direct increment scan line

i.e.

$$m = \frac{y_{k+1} - y_k}{x_{k+1} - x_k}$$

So $y_k = y_{k+1} - 1$

& $x_{k+1} = x_k + 1/m$ ($1/m$ = inverse slope)

(& we use integer values only)

- **Sorted Edge Table (SET)**

  polygon edges cross the scan line.

  It have — maximum y value
  → x intercept
  → inverse slope

  (y_{max}, x_{min}, $1/m$)

- **Active Edge list (AEL)**

  List of all polygon edges that intersect the current scan lines.

  It checks edge table, Add newly Active edge.
  Remove expired edge.
  Calculate x intersect
  Sort x values. then fill.

• Inside - Outside Test

Determine if a point is inside a polygon or not.

methods -

(1) Odd-even Parity Rule

(2) Non-zero winding number Rule

1) Odd-even Rule

a point inside a polygon if a ray drawn from that point crosses the polygon boundry an odd number of times.

If it crosses an even number of times; point is outside.

Algo

1. Select point p
2. Draw a ray p to outside of polygon
3. Count polygon boundray intersect
4. Count odd → inside
5. Count even → outside

2) Non-zero winding Number Rule

Winding a number of times a polygon boundary winds around a given point considering the direction of traversal.

$W = 0 \Rightarrow$ Outside

$W \neq 0 \Rightarrow$ Inside.

$W = (\text{upward crossing}) - (\text{downward crossing})$

if boundary goes upword +1

downword → -1

Eg:- Suppose a ray
2 upward
1 downward crossing

then $w = 2 - 1 = 1 \neq 0$

so point inside a polygon.

Algorithm select

1. Given point P select
2. draw a ray using P
3. identify intersections of polygon boundary
4. determine dimension using
   upward → +1
   downward → −1
5. Sum contributions
   $w = \Sigma$ crossing contribution
6. w = 0 ? outside
   else Inside.

Boundary Fill Algorithm :-

It is a seed-fill algo that fills a region starting from an interior point & continues filling neighbouring pixels until a specified boundary color is encountered.

start from inside → fill neighbouring → stop at boundary pixels.

It requires

1) Seed Point => start point inside region (x, y)
2) Boundary color =>
3) Fill Color => interior color
4) working

Color(x, y) ≠ boundary color

& Color (x, y) ≠ fill color

then Color(x, y) = fill color

[otherwise infinite recursion]

Algo

```
BoundaryFill 8(x, y, fillColor, boundaryColor)
{
 currentColor = getpixels(x, y)
 if (currentColor != boundary Color
    And currentColor != fillColor)
 {
  setpixel(x, y, fill Color)
 BoundaryFill (x+1, y, fill Color, boundaryColor)
 BoundaryFill (x-1, y, fillColor, boundaryColor)
 BoundaryFill (x, y+1, fillColor, boundaryColor)
 BoundaryFill (x, y-1, fillColor, boundaryColor)
 }
}
```

Advantage

- simple & Boundary Based
- Useful for Colored region

Limitation

- Closed boundary required
- Recursion problem
- Boundary color should be known

* Flood-fill Algorithm

Flood-fill Algorithm fills a connected region by replacing all pixels of a specified interior (old) color with a new fill color, starting from a seed point.

seedpoint → Check old Color → Replace with fill Color → fill neighbours

→ It does not require any boundary color
→ It replace old color

Current pixel (x, y)
check color (x, y) == old Color
if yes the fill Color
No then Neighbour check.

Algorithm

```
floodfill (x, y, oldColor, fillColor)
{
  CurrentColor = getpixel (x, y)
  if (CurrentColor == oldColor)
    {
      setPixel (x, y, fillColor)
      floodfill (x+1, y, oldColor, fillColor)
      floodfill (x-1, y, oldColor, fillColor)
      floodfill (x, y+1, oldColor, fillColor)
      floodfill (x, y-1, oldColor, fillColor)
    }
}
```

* Character Generation :-

Character generation is the process of producing & displaying alphanumeric characters & symbols on a computer graphics display.

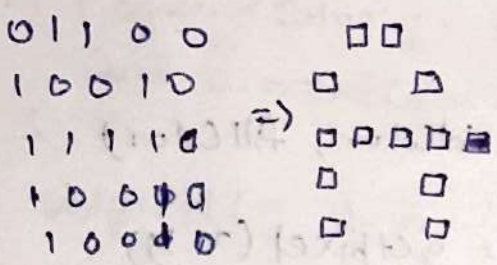
Basic methods

1) Bitmap (Raster) method 2) Stroke/Vector method

1) Bitmap method

Character is stored as pixel pattern.

Computer decides internally which pixel should on/off representation.



Stored in memory area.

Advantage

- Fast display & simple Implementation
- Useful for fixed size font

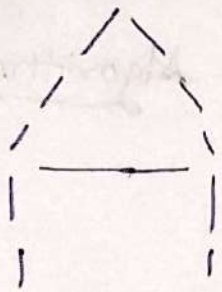
Limitation

Not suitable for variable size font

2) Stroke Method

Character is stored as set of line/strokes set.

move to (x_1, y_1)
Draw to (x_2, y_2)
Move to (x_3, y_3)
Draw to (x_4, y_4)
⋮



Advantage

Flexible ge representation
Easily scalable
Geometric Representation

Limitation

Complex to draw.

* Line Attributes To control appearance of Drawn Line

Line Attributes
1. Line type / Style
2. Line width
3. Line Color

1) Line type

Solid line ——————————

Dashed line — — — — —

Dotted line .....

Dash-dotted line —.—.—.—

according to pattern.

2) Line width

Thin Line ——————————

Thick line ═══════════

additional pixel draw to increase the line width.

3) Line Color

Desired color assign to line

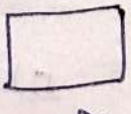
* Area-fill Attribute

properties used to control the appearace of the interior of a filled graphical object

Appearance = fill style + fill Color + fill Pattern

- **fill style** How object interior filled

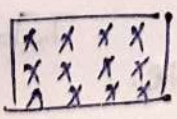
a) Hollow fill (empty) — no fill



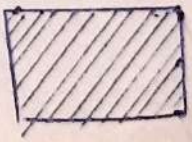
b) Solid fill — full fill



c) Pattern fill — dot / grid / line design



d) Hatch fill — parallel line fill



- **fill Color** Specify interior color of object.
- **Pattern fill** predefined pattern repeats in this

Suppose pattern

```
X .
. X
```

=>

```
X . X . X . X .
. X . X . X . X
X . X . X . X .
. X . X . X . X
```

Application of Area-fill Attributes

- CAD Drawing, maps, Bar charts, games, Technical diagram & Architectural Drawing.

* Character Attributes

Character attributes are the properties, used to control the appearance & orientation of character displayed on a graphic screen.

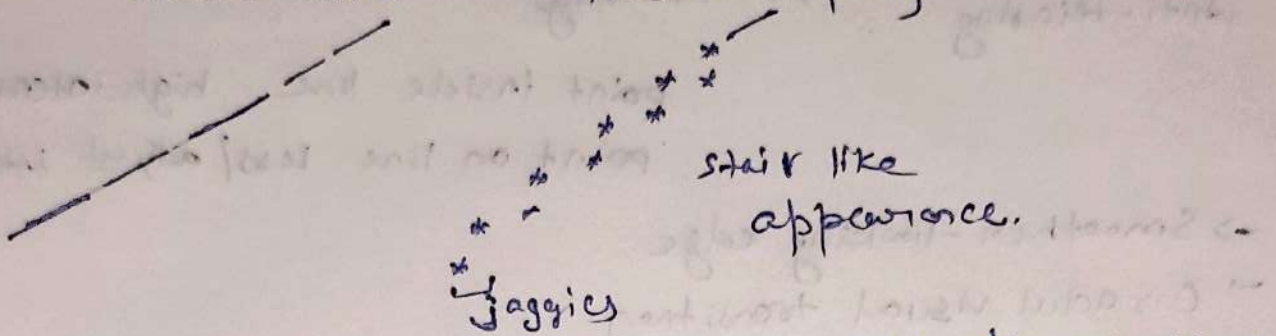
Main Attributes

1) Character size — Height + width.
   Represented as (W, H)
2) Character font/Style
   - Normal (visual design)
   - Bold
   - Italic
3) Character color
   - highlighting
   - warnings
   - Labels

   display color of character.
4) Orientation
   - Horizontal
   - vertical
   - & angle rotate.
5) Character Alignment
   - Left, center & Right
   - Top, bottom & center.

* Aliasing

Aliasing is the distortion or jagged appearance of graphical objects caused by representing continuous objects using a discrete set of pixels



Reason → Continuous image with discrete pixels

It can be seen in

1) Diagonal lines
2) Circle or Ellipse (Curved)
3) Text/Characters
4) Small Graphics

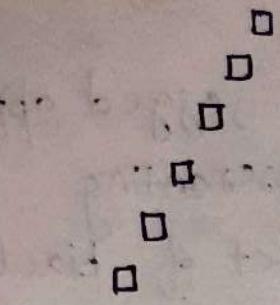
Higher Resolution → Less visible Aliasing.

Effects

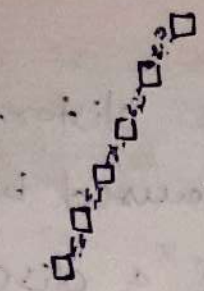
1) Rough or jagged graphics
2) Less smooth
3) Less visual quality.

* Anti Aliasing

Anti-Aliasing is a technique used to reduce the jagged or stair-step appearance of lines, curves & other graphical objects on a raster displ



without
Anti-Aliasing



with
Anti Aliasing

Edge pixel Coverage

↓

adjust pixel Intensity
/Color

point inside line high intensity
point on line less/ adjust intensity

-> Smoother-looking edge
-> Gradul visual transition
-> Reduce stair step appearance.